MAILDESK — EMAIL MANAGEMENT SYSTEM

Integrated OOPS and DBMS Project Report

Submitted by

_____ – _____

_____ – _____

_____ – _____

of

BACHELOR OF ENGINEERING

in

Department Name: _____

INSTITUTION: _____

(Institution / autonomous status, if applicable)

[INSTITUTION LOGO]

November 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled “MAILDESK — EMAIL MANAGEMENT SYSTEM” is the bonafide work of the students whose names and registration numbers are entered below. The work was completed as an academic project integrating Object Oriented Programming in Java and Database Management Systems.

The report describes the design, implementation, database structure, interface, and observed outputs of the submitted application. The student team has prepared the work for academic assessment and has identified the implemented functions and the known limitations of the project.

STUDENT DETAILS AND SIGNATURES

NAME OF STUDENT	REGISTER NUMBER	SIGNATURE
_____	_____	_____
_____	_____	_____
_____	_____	_____

INTERNAL FACULTY

Ms.DHANALAKSHMI K
Assistant professor
Object oriented programming in java
Artificial Intelligence and Data Science

Ms.DHANALAKSHMI K
Assistant professor
Database management systems
Artificial Intelligence and Data Science

Date of submission / practical examination: _____

INTERNAL EXAMINER	EXTERNAL EXAMINER
Signature: _____	Signature: _____
Date: _____	Date: _____

ABSTRACT

Maildesk is a Java desktop email-management system that brings common Gmail tasks into one focused workspace. A user can connect through Google OAuth, browse Inbox, Sent, and Gmail Spam, search the locally cached sender, recipient, subject, and body, read messages, and compose plain-text email. The Swing application uses an application service boundary so the interface, Gmail integration, local spam analysis, and persistence can be understood as separate parts.

The project's distinctive feature is an explainable second opinion for suspicious messages. A deterministic Java rule classifier assigns a score from visible signals such as blocked senders, suspicious phrases, many links, and urgent subject wording. The interface presents reasons with the score so the mailbox owner can review and correct the decision. A score is a rule score, not a probability. Gmail's own Spam label remains distinct from Maildesk's locally suspected messages. A saved user correction is retained per account.

For predictable demonstrations, the live connection and refresh retrieve up to six recent messages from each of Inbox, Sent, and Gmail Spam, deduplicate message IDs, and cap the local sample at 18 unique messages. The application stores accounts, messages, recipients, labels, spam results, feedback, and blocked senders in MySQL through JDBC. Composite account/message keys scope cached records to an account. OAuth tokens and database passwords remain in local configuration and are not stored in the message tables.

The separate demo launcher uses fictional in-memory messages. It does not connect to Gmail or MySQL, use the network, or deliver real email. The project uses Java 17 or newer, Swing, Maven, the Gmail REST API, Google OAuth, and MySQL 8. The report documents its flow, use cases, data model, modules, source-level implementation, screenshots, setup, and verification coverage. Attachments, rich text, scheduled synchronization, broad public OAuth distribution, and machine-learning classification are future work, not current features.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION: problem statement, project overview, purpose, scope	5–8
2	FUNCTIONALITIES PROVIDED	9
3	SYSTEM FLOW DIAGRAM: dataflow, use cases, ER model, architecture	10–14
4	MODULE DESCRIPTION: workspace, authorization, mailbox, messages, spam, demo	15–19
5	IMPLEMENTATION: Java contracts, Gmail, spam rules, JDBC, setup, verification	20–32
6	CONCLUSION	33
7	REFERENCES	34
A	COMPLETE SOURCE CODE (COMPANION LISTING)	35
B	APPLICATION OUTPUT AND SCREENSHOTS	36–40

Student names, register numbers, department, institution name and logo, and submission / practical-examination date are left for completion. Faculty details are shown on the bonafide certificate. No personal or institutional details from the supplied example have been copied into this report.

1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Email users often move between separate mailbox views to find recent work, inspect a message, compose a reply, and check suspicious requests. A basic spam filter can hide a message or provide little explanation for its decision. When users cannot see the reason, they have less confidence in the result and may have difficulty correcting a false positive.

A student-scale email management application should combine the common tasks in a clear interface while keeping the user involved in security decisions. It also needs a reliable way to preserve a small mailbox cache in a relational database without turning the project into a full Gmail mirror.

Maildesk addresses this problem with a focused desktop workspace, a bounded recent-message sample, MySQL persistence, and an explainable local spam review. The user sees matched warning reasons and can mark a received message as spam or not spam. Gmail's own Spam folder and Maildesk's local suggestions remain separate views.

The difficulty is not simply that a mailbox contains many messages. The same mailbox combines routine notices, personal correspondence, receipts, work requests, and potentially deceptive messages. A user must locate relevant mail while deciding which content deserves caution. A useful student project should therefore demonstrate both ordinary message handling and a transparent review mechanism. It should also explain exactly which data is local, which action reaches Gmail, and how the user can correct a recommendation.

1 - INTRODUCTION

1.2 PROJECT OVERVIEW

Maildesk is a Java Swing desktop client connected to Gmail through Google's OAuth flow and Gmail REST API. Its main window presents a navigation rail, mailbox list, search controls, a message reader pane, and a compose form. JDBC connects its application cache to a local MySQL database.

The core service is exposed through an EmailService interface. GmailEmailService coordinates remote Gmail actions with the local store; DemoEmailService implements the same interface using fictional in-memory messages. Because both implementations share a contract, the interface can be demonstrated without credentials, network access, or a running database.

On live startup, the system requests six recent items from Inbox, Sent, and Spam, deduplicates IDs, then stores no more than 18 unique messages. Local spam analysis attaches a score and reasons to received messages. Search covers the cached sample. Compose sends plain-text email after validation. The database persists message data, Gmail labels, spam explanations, feedback, and blocked senders per account.

The application has two deliberate execution paths. The live path uses the user-authorized Gmail account and MySQL. The presentation path uses deterministic fictional content held only in memory. Both paths use the same EmailService interface, which keeps the UI behavior consistent while making the demo safe and independent of internet access. The report treats these paths separately wherever a screenshot or test could otherwise be mistaken for a live mailbox action.

1 - INTRODUCTION

1.3 PURPOSE AND OBJECTIVES

The purpose of Maildesk is to demonstrate an end-to-end Java and DBMS application that makes email work easier to understand. The system centralizes reading, searching, writing, and review while showing how an object-oriented service layer can connect a Swing interface to both an external API and a relational database.

Project objectives

- Connect a Gmail account by using Google OAuth in the system browser; do not collect the mailbox password.
- Show Inbox, Sent, Gmail Spam, and locally Suspected Spam in a consistent desktop interface.
- Search the cached sender, recipient, subject, and plain-text body.
- Validate a plain-text draft before requesting a Gmail send.
- Persist account-scoped messages, recipients, Gmail labels, spam reasons, and user corrections through JDBC.
- Explain why the local rule classifier suggested review and allow the user to correct it.
- Provide a fictional offline demonstration that is safe to show without connecting to a real account.

These objectives provide a practical basis for evaluating the result: authorization should be explicit; message retrieval should be bounded; data access should remain account-scoped; remote changes should follow a user action; and suspicious-message recommendations should show their reasons. The team can demonstrate each objective by tracing a user action through the Swing panel, service method, provider or database adapter, and visible result.

1 - INTRODUCTION

1.4 SCOPE

CURRENT PROJECT SCOPE	OUTSIDE THE CURRENT BUILD
Gmail Desktop OAuth; Inbox, Sent, Gmail Spam; recent-message cache; local search.	Hosted multi-user mail service or a complete replacement for Gmail.
Read, send plain-text, refresh, mark spam / not spam.	Attachments, rich-text editing, and scheduled synchronization.
MySQL persistence via JDBC and one active connected account.	Enterprise administration or public multi-account hosting.
Transparent deterministic spam rules and account-specific feedback.	Machine-learning classification or a probability-calibrated risk score.
Offline demo with fictional, in-memory mail.	Real delivery from demo mode or database persistence in demo mode.

The live application uses Google permissions configured for the Gmail API. Distribution to a broad public audience may require the applicable Google consent-screen and OAuth verification work. The Windows package contains the application runtime, but each live installation still requires local MySQL and a user-provided Desktop OAuth client.

Scope is also a statement of what the team can support reliably. A small recent-message cache keeps the report and classroom demonstration reproducible, while the offline mode avoids requiring a reviewer to grant account access. Features such as full mailbox synchronization, attachment scanning, multiple simultaneous accounts, and machine learning would require additional design, privacy analysis, quota handling, and user testing, so they are documented as future work.

2 - FUNCTIONALITIES PROVIDED

Maildesk offers a compact set of workflows that can be explained and demonstrated from one workspace. A live mailbox owner signs in through Google, then chooses one of the four mailbox views. The interface updates the list and reader pane without replacing the connected account.

FUNCTION	USER VALUE	BOUNDARY
Connect Gmail	Use Google sign-in and load recent mail.	Requires configured Desktop OAuth credentials and consent.
Inbox / Sent / Gmail Spam	Read common mailbox categories.	Gmail labels identify provider folders.
Suspected Spam	Review the local rule model’s suggestions.	Separate from Gmail’s Spam folder.
Search	Find a message in the cached sample.	Does not search every remote Gmail message.
Compose	Send a validated plain-text message.	No attachments or rich formatting.
Correct spam	Save the user’s decision and update Gmail.	Only explicit user actions change Gmail labels.
Offline demo	Explore fictional data without external setup.	No network, MySQL, or real mail delivery.

Refresh replaces the local recent snapshot; it does not delete Gmail messages. Failed reads do not become an empty successful result. A local correction is associated with the account and may override the later automatic rule result.

3 - SYSTEM FLOW DIAGRAM

3.1 OVERALL SYSTEM FLOW

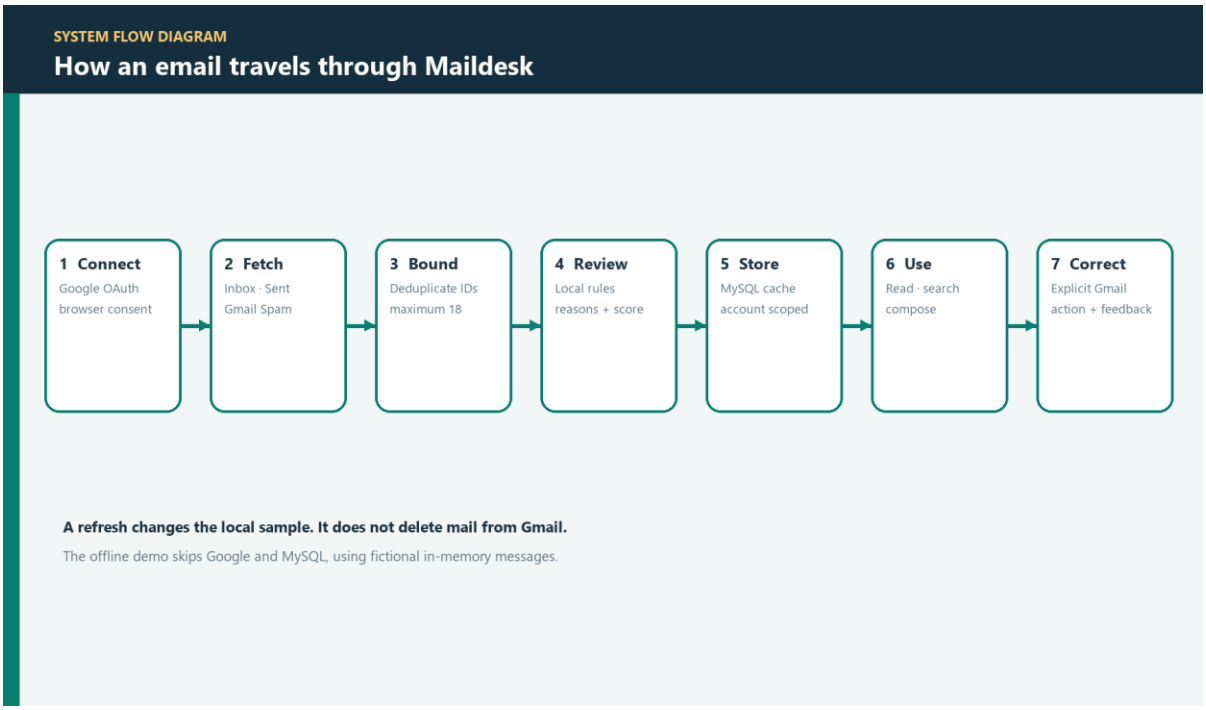


Figure 1: System flow from account connection through user review

The flow starts when a user connects Gmail. Maildesk requests up to six recent messages from each supported provider label. IDs returned from more than one label are deduplicated before the 18-item cap is applied. The service evaluates received messages locally and writes the snapshot and analysis into MySQL.

The person then reads, searches, or composes. Gmail state changes occur only after a corresponding user action. A spam correction is also saved to local feedback. In offline demo mode, the connection step is simulated and the data path remains in memory.

Each boundary has a defined responsibility. OAuth establishes identity and permission; Gmail supplies provider messages and receives explicit user changes; the classifier adds a local review signal; the JDBC store persists a bounded cache and feedback; the UI presents the combined state. The demo mode replaces the live integrations with in-memory behavior while preserving the workflow. A failure at one boundary is surfaced as a failed action rather than being silently converted into successful empty data.

3 - SYSTEM FLOW DIAGRAM

3.2 DATAFLOW DIAGRAM

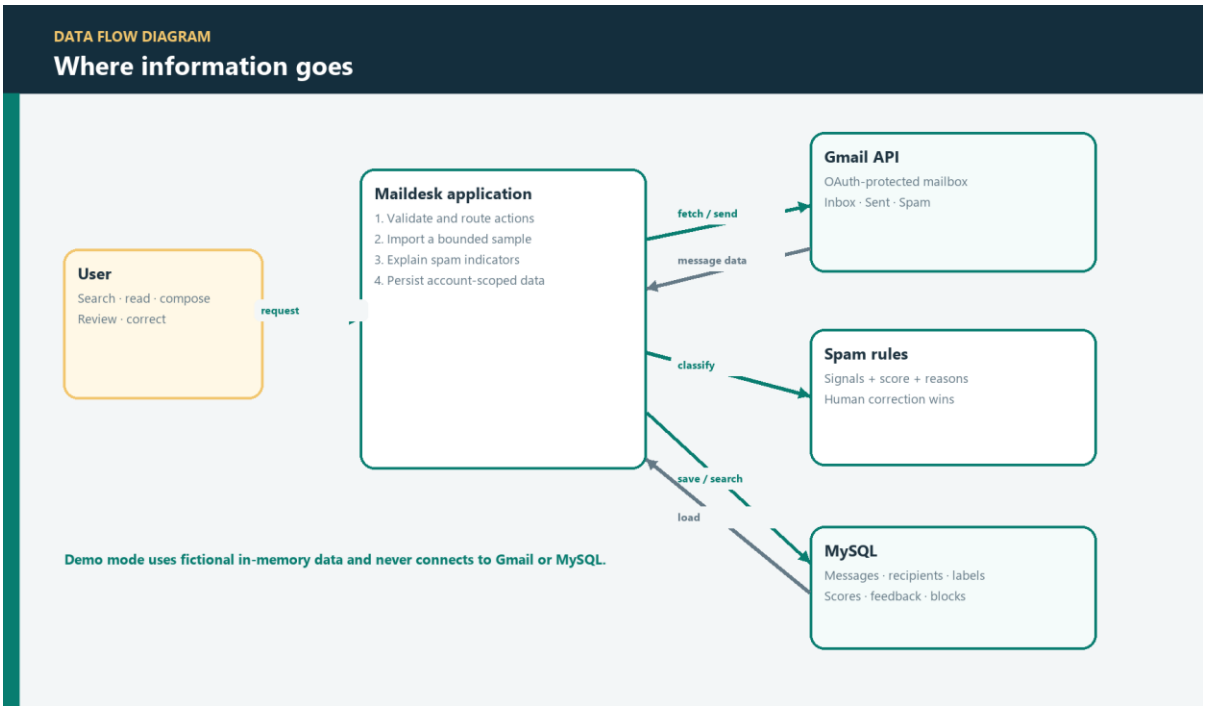


Figure 2: Data flow between the person, application, Gmail, classifier, and MySQL

The Swing interface accepts searches and explicit actions. The application service validates the selected mailbox or draft and routes requests to the Gmail gateway or the database store. Gmail returns message metadata and body text; the local classifier derives a score and explanation from the message fields and account-specific blocked-sender list.

The database stores a cache and the results needed for the local UI. OAuth tokens remain in a local protected token store rather than in the email tables. The demo implementation bypasses all external boxes and reads fictional fixtures from memory.

The data flow distinguishes message content from authorization credentials. Message metadata and plain-text content pass through the service into the local cache; OAuth tokens are kept in a separate local token directory. Search is limited to the loaded sample. The database does not act as a public server and is not exposed to other accounts. This separation helps explain why neither a message's Gmail password nor an OAuth secret is stored in the relational message rows.

3 - SYSTEM FLOW DIAGRAM

3.3 USE CASE DIAGRAM

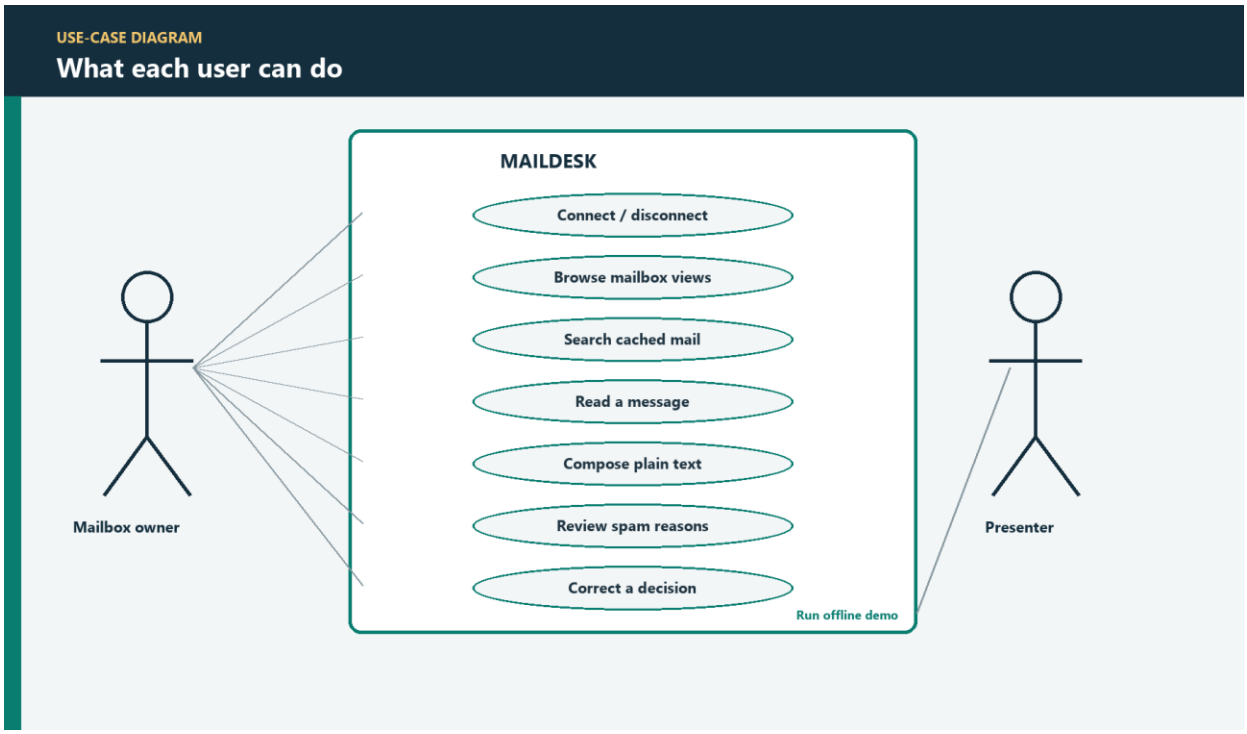


Figure 3: Mailbox owner use cases and the presenter's demo entry

ACTOR	USE CASE	DESCRIPTION
Mailbox owner	Connect / disconnect	Authorize a Gmail account or end the local session.
Mailbox owner	Browse / search	Choose Inbox, Sent, Spam, or Suspected Spam; search the cache.
Mailbox owner	Read / compose	Open a cached message or compose a validated plain-text message.
Mailbox owner	Review / correct	Read rule reasons and mark a received message spam or not spam.
Presenter	Run demo	Explore fictional content without Gmail or database setup.

The primary actor is the mailbox owner because every live action is performed under that person's authorization. The presenter actor has a narrower role: start offline demo mode and show fictional output. Use cases that change Gmail—sending, marking spam, clearing spam, and marking read—are distinguished from local actions such as opening a view or searching the current sample. This makes the user's authorization and intent visible in the requirements.

3 - SYSTEM FLOW DIAGRAM

3.4 ENTITY RELATIONSHIP DIAGRAM

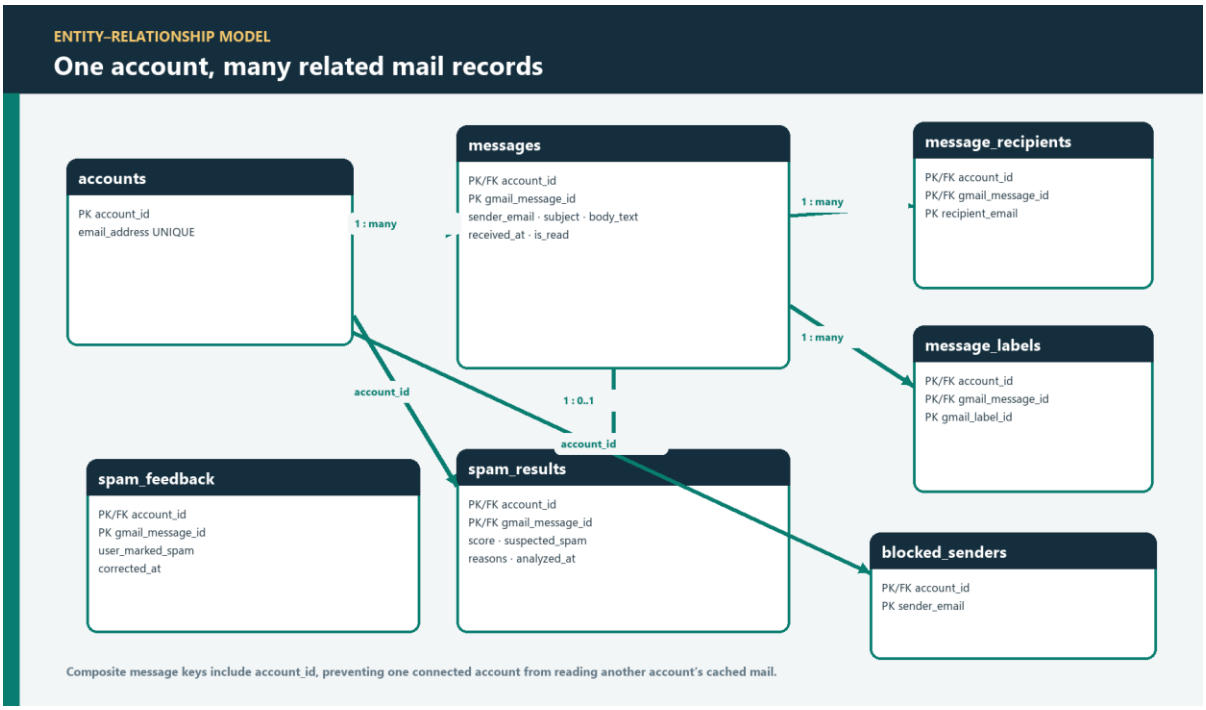


Figure 4: MySQL entities and account-scoped key relationships

The accounts table identifies each connected mailbox. Messages use the composite key (account_id, gmail_message_id). Recipient and label rows reference that key. Spam results are stored per message, while feedback and blocked-sender rows are associated with the account. This structure keeps repeated recipients and Gmail labels out of a single overloaded message column.

Foreign keys enforce parent-child relationships. The composite message key ensures lookups and writes are scoped to the active account. sql/schema.sql contains the full table definitions and constraints.

The schema resolves one-to-many relationships explicitly. One account owns many messages; one message can have several recipients and Gmail labels; one message can have a current spam analysis and a user feedback record. Composite keys and foreign keys provide referential integrity, while account IDs prevent a lookup from treating two users' identical Gmail message IDs as the same local row. The complete DDL is reproduced in Appendix A.

3 - SYSTEM FLOW DIAGRAM

3.5 SOFTWARE ARCHITECTURE

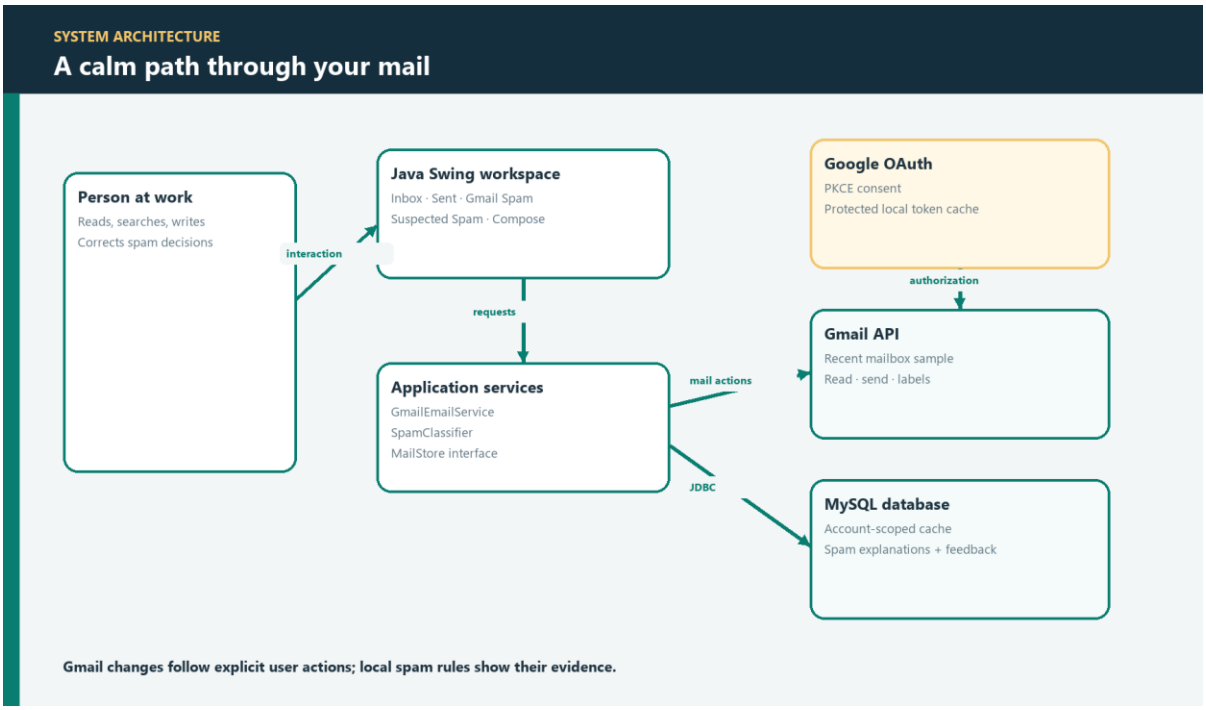


Figure 5: Swing, service, Gmail, OAuth, spam analysis, and persistence layers

The Swing UI depends on EmailService rather than on GmailApiClient or JdbcMailStore directly. GmailEmailService is the live implementation. DemoEmailService uses the same application boundary with fictional data. This separation allows interface behavior to be reviewed without granting real Gmail permissions.

The Gmail package owns OAuth and REST details. The storage package owns the MailStore contract and its JDBC implementation. SpamClassifier is a deterministic local component. Model classes carry immutable message and result values between these layers.

Dependency direction is chosen to keep the interface replaceable. The UI calls the service contract; GmailEmailService and DemoEmailService implement that contract. The live service coordinates GmailGateway and MailStore, while the spam component computes an explanation from domain data. This design limits provider-specific HTTP details to the Gmail package and SQL details to storage, reducing the chance that a screen handler contains persistence logic.

4 - MODULE DESCRIPTION

4.1 MODULE OVERVIEW AND ACCOUNT ACCESS

Maildesk is organized into a welcome view, mailbox workspace, reader, compose form, spam-review panel, application service, Gmail adapter, classifier, and JDBC store. Each module has a defined responsibility. The Swing layer presents actions and results; the service layer coordinates the workflow; provider and database details remain behind integration boundaries.

Welcome and account connection

The welcome view explains the product before requesting account access. Its primary action begins Google authorization in the system browser; the offline-demo action opens fictional content without requiring credentials. The app does not ask the user to enter a Gmail password. A status message distinguishes missing setup, active authorization, and recoverable errors.

Google authorization

The live path uses a Google Desktop OAuth client and Authorization Code with PKCE. Maildesk generates a random state and verifier, binds a temporary callback to 127.0.0.1, validates the returned state, and exchanges the code over HTTPS. Tokens are saved locally outside the message tables. The user must still enable the Gmail API and configure the Google consent screen and test-user list.

4 - MODULE DESCRIPTION

4.2 MAILBOX NAVIGATION AND RECENT SAMPLE

The workspace presents Inbox, Sent, Gmail Spam, and Suspected Spam as distinct views. The first three correspond to Gmail labels. Suspected Spam is generated locally by Maildesk. The message table shows sender or recipient, subject, and time; a selected message is opened in the reader pane. Selection uses a stable Gmail message ID rather than a visible row number, so sorting does not change the message targeted by an action.

Bounded refresh

Connection and refresh request up to six recent messages from each of Inbox, Sent, and Spam. The Gmail adapter deduplicates IDs that appear under multiple labels, and the service caps the combined local sample at 18 unique messages. Search operates on this cache and is not presented as remote, account-wide Gmail search.

Refreshing replaces only Maildesk’s local sample. It never deletes Gmail messages. Database snapshot replacement runs transactionally; if a provider read or local write fails, the previous cache is not reported as a successful empty result. This keeps the displayed state consistent with what the application actually confirmed.

4 - MODULE DESCRIPTION

4.3 SEARCH, READER, AND COMPOSE

Search and message reader

Search checks sender, recipients, subject, and cached plain-text body. An empty query restores the selected mailbox. The reader displays sender, recipient, subject, date, body, labels, and any spam explanation. Message content is rendered as text rather than interpreted as HTML. A missing body is handled as an incomplete message detail and does not prevent the remaining fields from appearing.

Compose and send

The compose panel collects one recipient, a subject, and a plain-text body. Validation rejects malformed recipient input, a blank or multi-line subject, and an empty body before a network request is made. The service repeats validation so correctness does not depend on the Swing form. The Gmail adapter builds a UTF-8 MIME message and sends only after the user chooses Send.

During a pending operation the interface disables duplicate submission and reports success or a recoverable failure. Sends are not automatically replayed after ambiguous network errors because the first request may already have reached Gmail. Attachments, rich text, multiple recipients, and synchronized drafts are outside this version.

4 - MODULE DESCRIPTION

4.4 EXPLAINABLE SPAM REVIEW

Maildesk's classifier is a deterministic second opinion. It adds documented weights for a blocked sender, matched suspicious phrases, numerous links, and urgent subject wording. The configured threshold determines which messages appear in the local review queue. The score is a rule score, not a probability and not a machine-learning prediction.

SIGNAL	WEIGHT / CONDITION	EXPLANATION SHOWN
Blocked sender	+50	Sender matches the account's blocked-sender list.
Suspicious phrase	+20 per match, at most three	The matching phrase is identified.
Many links	+15 for five or more	Message contains an unusually large number of links.
Urgent subject	+10	Subject contains configured urgency language.
Review threshold	50 points	Display the local Suspected Spam suggestion.

Each contributing signal is retained as a reason. The user can inspect that evidence before taking action. Gmail's own Spam label remains separate from the local suggestion, so displaying a message in the review queue does not by itself change the provider mailbox.

4 - MODULE DESCRIPTION

4.5 USER CORRECTION AND OFFLINE DEMONSTRATION

User correction

When the mailbox owner selects Mark as spam or Not spam, the live service requests the matching Gmail label change and records account-scoped feedback in MySQL. Automatic analysis, user correction, and blocked-sender preferences are stored separately. If the provider update fails, the interface reports the failure instead of claiming that Gmail changed. No send or label update is automatically replayed.

Offline demonstration

DemoEmailService implements the same EmailService contract using invented messages held in memory. It supports navigation, search, reading, compose simulation, and spam correction. The banner states that no message is sent, no network is used, and no MySQL data is written. Restarting the app reloads the fixed example set.

Appendix B contains the application outputs captured in this mode. Addresses use reserved example domains; screenshots document the interface and its behavior, not a real account or a delivered email.

5 - IMPLEMENTATION

5.1 SOURCE ORGANIZATION AND SERVICE CONTRACT

SOURCE AREA	RESPONSIBILITY
ems.ui	Swing frames, panels, table model, compose form, and visual theme.
ems.service	Shared EmailService contract and live workflow coordination.
ems.gmail	OAuth, token handling, Gmail REST, MIME construction, and parsing.
ems.storage	MailStore boundary, JDBC implementation, and schema initialization.
ems.model / spam	Account, mailbox, message, spam result, and rule classifier.
src/demo/java	Offline demo launcher and fictional in-memory service.
src/test/java	Java verification and fixture sources.
sql, scripts, pom.xml	Database DDL, Windows tooling, and Maven build configuration.

The Swing UI depends on EmailService and does not construct SQL or Gmail HTTP requests. GmailEmailService and DemoEmailService implement the same method contract. This permits the same navigation and compose workflow to run against either the live provider or fictional demo records. Application exceptions carry recoverable messages to the UI without exposing credentials.

Appendix A, supplied as the companion source listing, reproduces all Java implementation and test files, SQL, Windows scripts, Maven configuration, and wrapper / CI configuration.

5 - IMPLEMENTATION

5.2 DOMAIN MODEL AND SWING RESPONSIVENESS

Account identifies the locally active Gmail address and database key. EmailMessage carries the account ID, Gmail message ID, sender, recipients, subject, plain-text body, received time, and Gmail labels. Mailbox names the visible folder. SpamResult keeps the numeric rule score, review flag, and explanation list. These types prevent screen code from depending directly on database result sets or raw provider JSON.

Message collections are copied when model objects are constructed so a caller cannot mutate the value through a retained list. Gmail message identity is stable across refreshes and is used to address open and label operations. Local account identity accompanies each stored message, preventing IDs from different accounts from being conflated.

MailAppPanel uses SwingWorker for operations that may wait for Gmail or MySQL. The worker performs service work in the background and returns UI updates to the event-dispatch thread. A busy state disables duplicate actions and is cleared on both success and failure. The reader and compose panels are reusable Swing components; MailTableModel supplies a consistent row view for each mailbox.

5 - IMPLEMENTATION

5.3 GOOGLE OAUTH AND GMAIL OPERATIONS

GmailAuth implements the native-app authorization flow with PKCE. It creates a state value and high-entropy verifier, derives an S256 challenge, starts a temporary loopback listener on an available port, and opens the Google authorization page. The callback checks state and timeout conditions before the authorization code is exchanged. The server is closed in all completion paths.

GmailApiClient isolates HTTPS request construction, access-token attachment, response checking, and message conversion. GmailGateway provides a smaller service-facing boundary. The client reads the authorized account profile, lists messages by label, fetches message details, sends a MIME message, and modifies labels after explicit user actions.

GMAIL OPERATION	MAILDESK USE
Profile lookup	Identify the authorized Gmail address.
List and get	Load bounded IDs, metadata, labels, and plain-text content.
Send	Submit a user-composed plain-text MIME message.
Modify	Mark read or apply / remove Spam after a user action.

Transient read requests use bounded retry behavior. Sends and label updates are not automatically replayed after errors because their completion may be ambiguous. Tests use local HTTP fixtures; they do not send real messages.

5 - IMPLEMENTATION

5.4 BOUNDED IMPORT AND SPAM CLASSIFICATION

The import path requests at most six recent entries from each of INBOX, SENT, and SPAM. A linked set removes duplicate Gmail IDs, and GmailEmailService imposes the final limit of 18 messages before replacing the local snapshot. The bounded sample reduces first-run work and makes the classroom demonstration easy to inspect; it is not a complete synchronization engine.

The classifier normalizes sender and message text, checks account-specific blocked senders, matches configured suspicious phrases, counts links, and evaluates urgency terms in the subject. Matched signals contribute fixed weights, and the classifier returns both the score and a human-readable reason list. The review threshold is a deterministic rule and carries no calibrated probability.

User feedback is kept distinct from automatic analysis. On later display, the saved decision may override the automatic suggestion for that account and message. A user can therefore see what the rules suggested and what they chose without the stored record misrepresenting either result.

5 - IMPLEMENTATION

5.5 DATABASE SCHEMA: ACCOUNTS AND MESSAGES

The accounts table uses an auto-generated local key and a unique email address. The messages table uses a composite primary key of account_id and gmail_message_id. Message rows hold sender, subject, body text, timestamp, and read status. Foreign keys enforce account ownership. The schema uses utf8mb4 to store international subject and body content.

The local key is included in the message primary key because Gmail message identifiers are meaningful within an authorized mailbox and should not be treated as globally unique across local accounts. This composite key also supports idempotent refresh: the same Gmail message updates the same account-scoped row.

The full CREATE TABLE statements, indexes, and constraints are reproduced in Appendix A. The schema initializer creates the project database and tables when the configured MySQL account has the required local privileges. A dedicated least-privilege database account is recommended for normal use.

5 - IMPLEMENTATION

5.6 NORMALIZED RECIPIENTS AND LABELS

A message may have several recipients and several Gmail labels. The message_recipients and message_labels tables store these values as child rows instead of delimiter-separated text. Each child table references the message's composite account and Gmail ID key. This keeps individual values queryable and prevents a child row from being attached to a message belonging to another account.

The child primary keys include the parent message key plus the recipient address or label ID. A unique key prevents duplicate child values during repeated refreshes. Foreign keys use cascading deletion so removal of an obsolete local cache row also removes its recipient and label rows. This is a local cache relationship only; it does not delete a Gmail message.

The normalized arrangement demonstrates relational modeling concepts used in the DBMS portion of the project: atomic values, clear one-to-many relationships, primary-key design, referential integrity, and controlled update behavior.

5 - IMPLEMENTATION

5.7 SPAM RESULTS, FEEDBACK, AND BLOCKED SENDERS

TABLE	KEY DATA	PURPOSE
spam_results	account, message, score, flag, reasons, analysis time	Latest automatic explanation.
spam_feedback	account, message, user decision, correction time	Mailbox owner’s saved correction.
blocked_senders	account, normalized email address	Account-specific classifier input.

These records are separated because a computed result is not the same thing as a user correction. spam_results is recalculated from message content and configured rules. spam_feedback records a decision made by the user. blocked_senders supplies an account preference that affects a later classification. This separation makes the review history easier to explain and query.

Repository methods use the account identifier together with the message identifier in lookups and updates. This prevents one account’s saved correction from affecting another account’s message and permits feedback to remain available even if an old message is not part of the current bounded sample.

5 - IMPLEMENTATION

5.8 JDBC PERSISTENCE AND TRANSACTIONS

JdbcMailStore implements MailStore using JDBC connections and prepared statements. It creates or resolves the account row, checks message ownership, writes message fields, replaces normalized recipient and label rows, stores the current spam result, and updates the local cache within transaction boundaries. Parameter binding keeps user-provided values separate from SQL syntax.

Snapshot replacement is transactional: the store disables auto-commit, applies related updates, and commits only after the whole operation succeeds. On a database exception it rolls back before returning an error. This avoids a half-refreshed state in which some new records appear while stale records remain as though the snapshot were complete.

DatabaseConnection reads configured connection values and exposes connections to the repositories. The application reports connection and schema errors with actionable setup guidance. MySQL availability is a separate live prerequisite; the offline demo deliberately bypasses JDBC.

5 - IMPLEMENTATION

5.9 CONFIGURATION AND DATA PROTECTION

AppConfig reads local properties and supports environment-variable overrides for database URL, username, password, OAuth credentials, and token directory. Example configuration is committed without secrets; actual credentials and token files remain local and are excluded from Git. OAuth client JSON and password values must never be copied into a screenshot or report source listing.

Gmail authorization is performed in the system browser and uses the Gmail modify scope required for reading and label changes. The application stores local OAuth tokens separately from messages and does not store the mailbox password. A user should use a dedicated local MySQL account with permission limited to the project database.

LOCAL VALUE	STORAGE LOCATION	HANDLING
Database login	app.properties or environment	Keep private; do not commit.
Desktop OAuth client	User-selected JSON file	Supply locally; not packaged.
OAuth tokens	Local token directory	Separate from message database.
Cached message data	Configured MySQL database	Bounded and account-scoped.

5 - IMPLEMENTATION

5.10 ERROR HANDLING AND RESPONSIVENESS

The application distinguishes missing OAuth configuration, denied access, provider errors, database unavailability, invalid user input, and unexpected local failures. Status messages describe the next action without exposing a token, password, or raw sensitive provider response. A provider read failure does not appear as a valid empty mailbox.

Network and JDBC work runs outside the Swing event-dispatch thread. The UI disables conflicting controls during an operation, restores them in a finally-style completion path, and keeps the prior signed-in state and cache if a refresh fails. A failed open clears stale reader content so it cannot be mistaken for the newly selected message.

Retry is bounded and limited to transient reads. Sends and label mutations are not retried automatically after an ambiguous result. This policy avoids duplicate delivery and repeated remote changes while keeping failure behavior understandable to the user.

5 - IMPLEMENTATION

5.11 BUILD AND WINDOWS PACKAGE

The Maven Wrapper builds the Java application using the JDK target declared in pom.xml. Production and demo launchers have separate entry points. The packaging script uses jpackage to create a Windows x64 application image with a bundled runtime and prepares a ZIP for distribution.

A live installation still requires a local MySQL server, a configured application database, and the user's own Google Desktop OAuth JSON. The package does not include OAuth secrets, token files, or database passwords. Users should extract the complete ZIP before running the executable so its bundled runtime and libraries remain available.

The source repository contains the build, setup-check, demo, and packaging scripts. Appendix A includes those scripts and the Maven configuration alongside the Java and SQL listings.

5 - IMPLEMENTATION

5.12 VERIFICATION AND ACCEPTANCE WALKTHROUGH

AREA	VERIFICATION APPROACH
OAuth	Local callback, PKCE verifier, state validation, timeout, and error cases.
Gmail API	Local HTTP fixtures for parsing, MIME, labels, and bounded retry.
Spam rules	Weights, threshold, reasons, blocked sender, and user override.
JDBC	Schema, account isolation, transaction rollback, sample replacement.
Swing UI	Navigation, search, validation, busy state, and failure display.
Live acceptance	Opt-in Google test account and local MySQL verification.

Automated checks use controlled local fixtures and do not send real mail. A live acceptance walkthrough is separate: configure a test account, connect Gmail, refresh the sample, compare the bounded count, open a message, mark one item for review, and inspect the corresponding account-scoped rows in MySQL Workbench.

An offline demo proves that the presentation flow works without external services; it does not prove that the OAuth client or a local database is configured.

5 - IMPLEMENTATION

5.13 LIMITATIONS AND FUTURE WORK

CURRENT LIMITATION	POSSIBLE NEXT STEP
At most 18 recent unique messages.	Add user-controlled pagination after quota review.
Plain-text compose with one recipient.	Design safe attachments and richer drafts.
Deterministic rules; no trained model.	Evaluate with a consented, labeled sample and measure false positives.
One active account.	Design stronger isolation before multi-account support.
Manual refresh.	Consider controlled sync and notification preferences.
Google app configured for student testing.	Complete applicable consent and OAuth review for broad distribution.

The current project does not claim machine-learning classification, complete mailbox synchronization, or public production readiness. Any future classifier should retain visible evidence and keep the mailbox owner in control. New synchronization, attachment, and account features should be described as implemented only after they are built and verified.

6 - CONCLUSION

Maildesk combines Java Swing, object-oriented service boundaries, Gmail OAuth and REST access, MySQL, and JDBC in one email-management application. The workspace supports mailbox navigation, local search, message reading, validated plain-text composition, and explicit label actions.

The report's principal project contribution is an explainable second opinion for suspicious messages. The rule score is accompanied by reasons, and a person can correct the result. Gmail's own Spam label stays distinct from Maildesk's local review queue. The bounded 18-message sample makes the classroom demonstration predictable and manageable.

A separate offline demo provides fictional messages for presentation and interface review. The live path requires a correctly configured Desktop OAuth client and local MySQL server. The project clearly identifies limits such as attachments, rich text, scheduled synchronization, machine learning, and production OAuth approval as future work.

Through this project, the team can demonstrate how an object-oriented Java application communicates with an external provider, protects account boundaries in a relational schema, and reports user-visible reasons for a security-sensitive suggestion.

The completed work also establishes a practical base for future development. The current modules make it possible to add controlled pagination, improved message composition, or additional review signals without merging provider, database, and screen responsibilities into a single class. Any expansion should preserve the existing principle that mailbox owners can inspect and correct suspicious-message suggestions.

The implementation should be presented as a working academic desktop application with defined boundaries: the live Gmail path depends on local Google OAuth configuration and MySQL, while the screenshot and classroom demo path uses fictional in-memory data. This distinction is part of the project's correctness and is reflected throughout the report and its appendices.

7 - REFERENCES

1. Maildesk source repository, setup guide, architecture documentation, SQL schema, and Windows packaging notes. https://github.com/Sanjay-AI-ML/email-management-work

2. Google for Developers. Gmail API documentation. https://developers.google.com/workspace/gmail/api

3. Google Identity. OAuth 2.0 for native and desktop applications. https://developers.google.com/identity/protocols/oauth2/native-app

4. Google for Developers. Gmail API authorization scopes. https://developers.google.com/workspace/gmail/api/auth/scopes

5. Oracle. Java SE documentation. https://docs.oracle.com/en/java/

6. MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

7. Project source files: src/main/java/ems, src/main/java/com/emailapp/spam, src/demo/java, sql/schema.sql, docs/SETUP.md, and docs/ARCHITECTURE.md.

8. User-provided reference document: EMPLOYEE DETAILS AND PAYROLL MANAGEMENT SYSTEM (academic report structure used as the formatting model).

9. Oracle. Java Swing documentation. https://docs.oracle.com/javase/tutorial/uiswing/

10. Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/

11. IETF. RFC 7636: Proof Key for Code Exchange by OAuth Public Clients. https://datatracker.ietf.org/doc/html/rfc7636

12. Google for Developers. Gmail API messages resource and users.messages methods. https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages

13. MySQL. Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

APPENDIX A — COMPLETE SOURCE CODE

This appendix is provided as a separate companion document so the project report remains exactly 40 pages while keeping all source listings readable. The companion contains every production and demo Java file, all Java test files, the MySQL schema, Windows scripts, Maven configuration, Maven Wrapper scripts, and the CI workflow.

SOURCE GROUP	FILES	CONTENTS
PRODUCTION JAVA	34	Complete Java source files.
DEMONSTRATION JAVA	2	Complete Java source files.
JAVA VERIFICATION SOURCES	13	Complete Java source files.
DATABASE SCHEMA	1	SQL DDL and constraints.
WINDOWS SCRIPTS	6	Build, run, check, and package scripts.
BUILD / WRAPPER / CI	5	Maven build, wrapper, and verification workflow.

Open Maildesk_Appendix_A_Source_Code.docx from docs/report or the linked project repository to read the complete listing. It is generated from the same source files as the project build. Appendix B, which follows, contains the application output screenshots.

APPENDIX B — APPLICATION OUTPUT

The following outputs were captured from Maildesk's offline demonstration mode. All people, addresses, subjects, message bodies, and dates shown are fictional. The demo does not connect to Gmail or MySQL, use the network, save messages to disk, or deliver real email. These pages document the appearance and behavior of the user interface only.

B.1 Welcome and demo entry

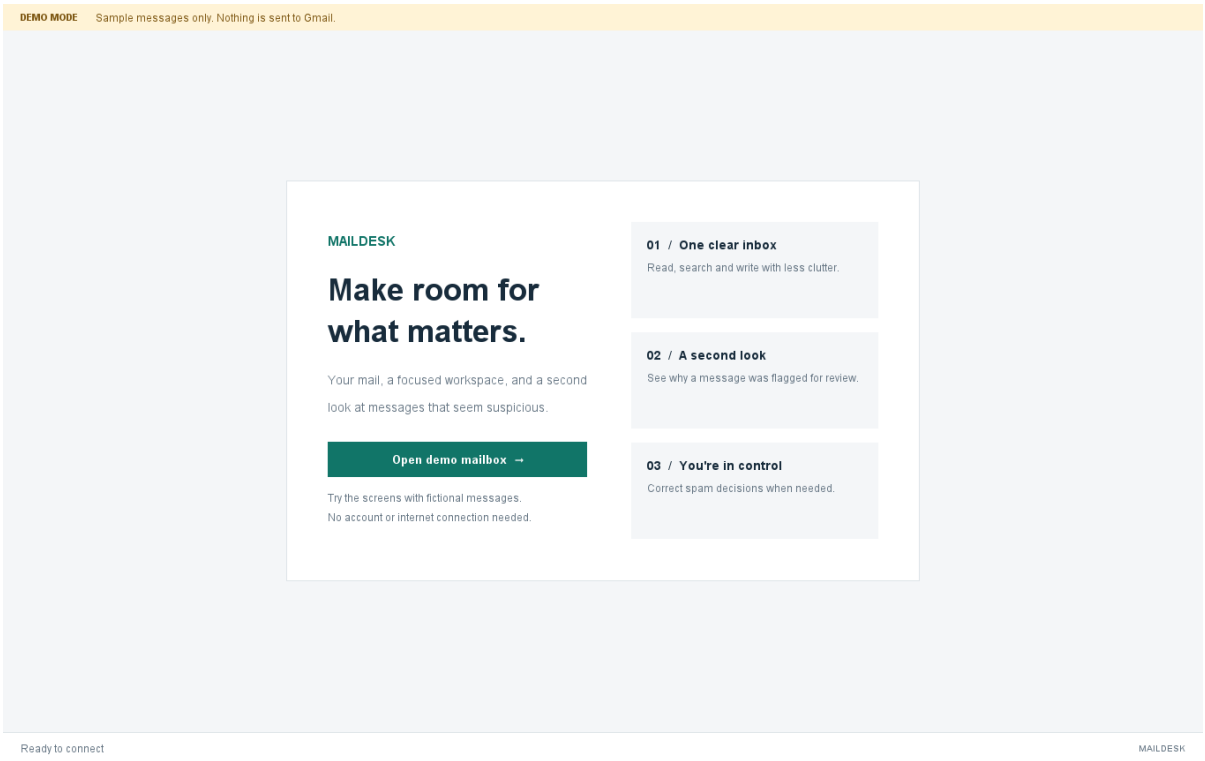


Figure 6: B.1 Welcome and demo entry

The welcome view introduces Maildesk and shows the entry point for the offline presentation. The Connect Gmail action in the live build opens Google authorization; the demo action opens fictional messages. The visible copy makes that distinction clear before the workspace is entered.

B.2 Inbox workspace

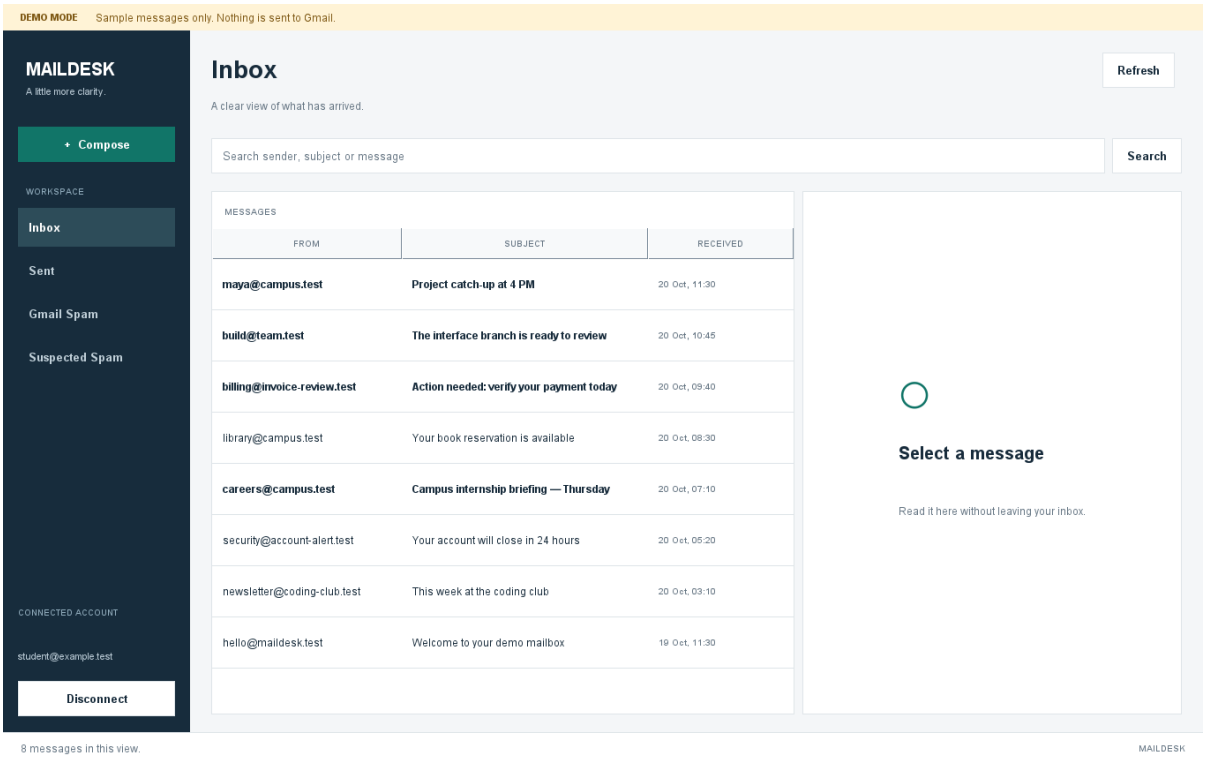


Figure 7: B.2 Inbox workspace

The workspace output shows mailbox navigation, search, the message table, selected-message context, and status controls. The rows are seeded example content. In a live session the same layout is populated from the bounded local cache retrieved through Gmail.

B.3 Message reader and explanation

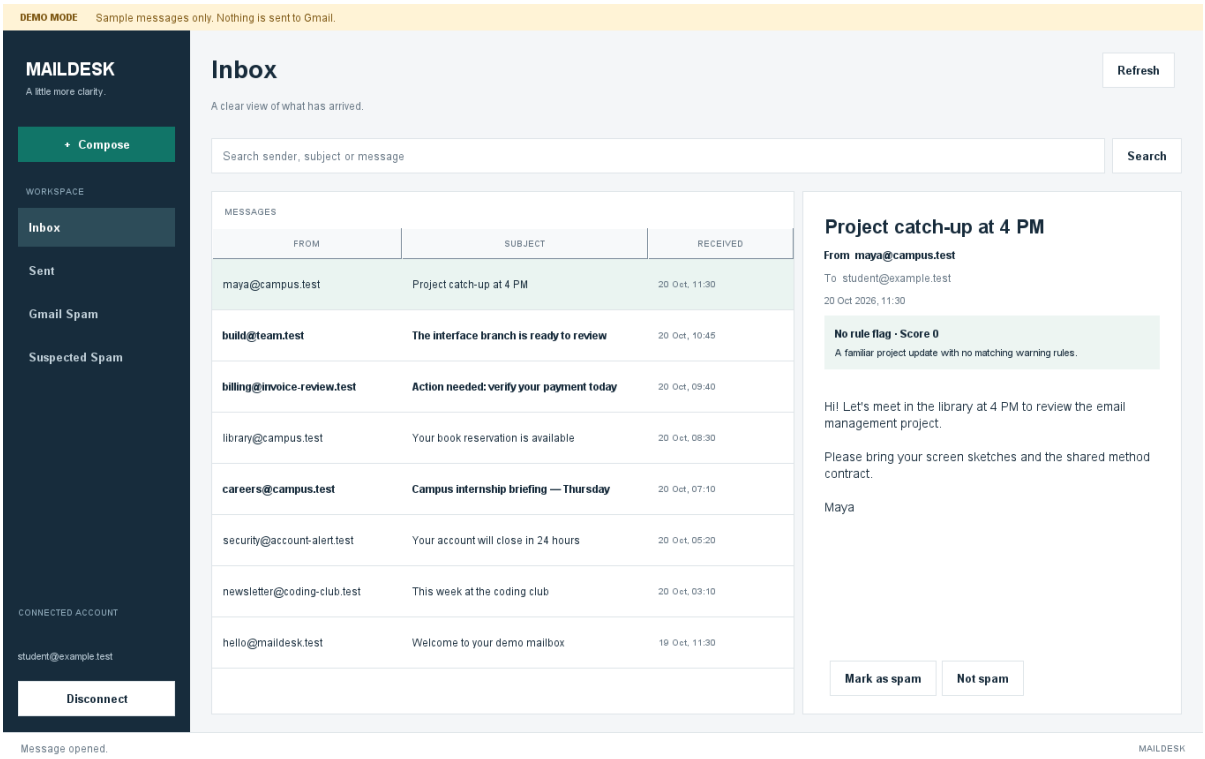


Figure 8: B.3 Message reader and explanation

The reader output shows message details and the local spam review card beside the mailbox list. Body content is displayed as plain text. A real live result depends on a successful authorized Gmail read; the image shown here is fictional demo data.

B.4 Plain-text compose form

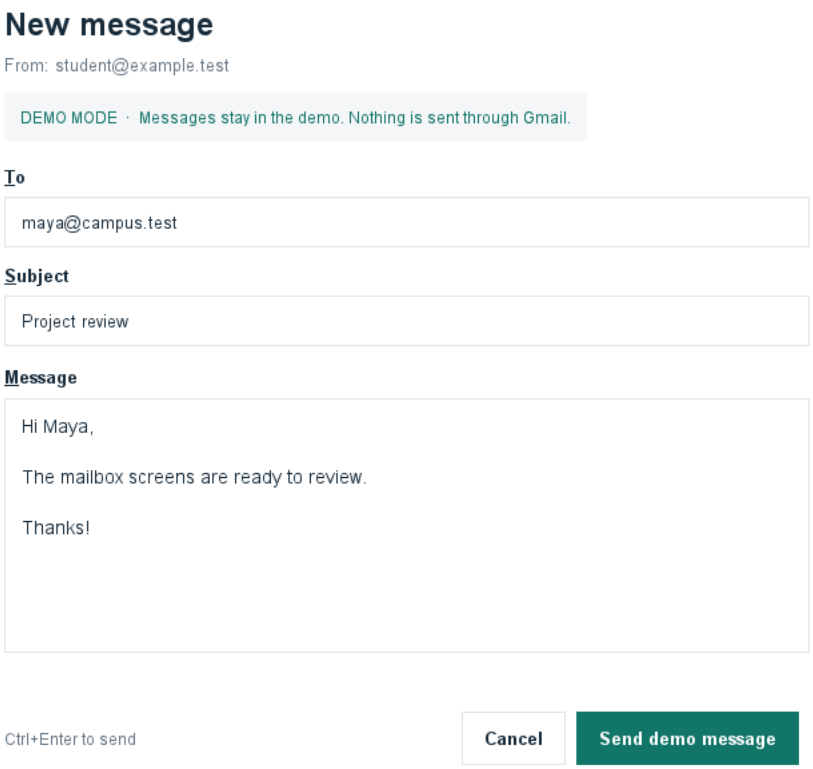


Figure 9: B.4 Plain-text compose form

The compose output shows recipient, subject, and body fields with the offline-demo notice. Validation is shared with the live path, but the demo records a simulated sent item in memory and never contacts Gmail.

B.5 Suspected Spam review

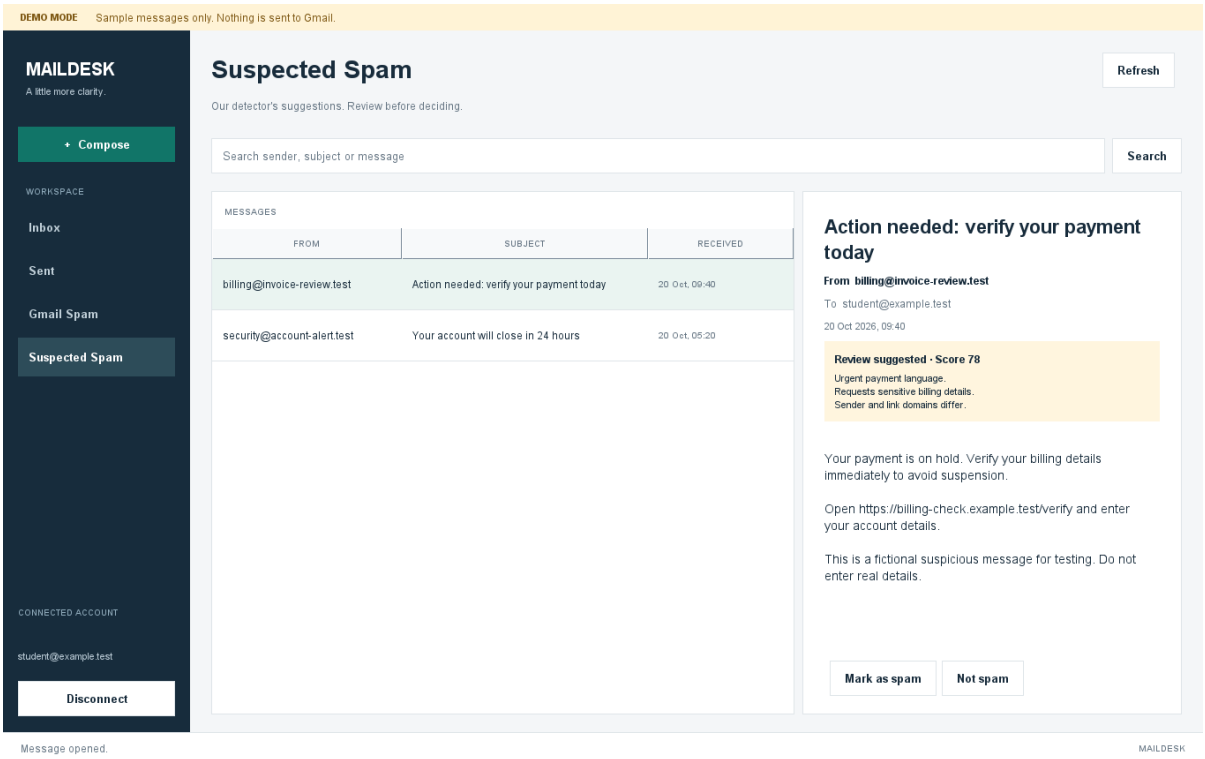


Figure 10: B.5 Suspected Spam review

This output displays a fictional suspicious message, its rule score, matched reasons, and user correction controls. It demonstrates the difference between Maildesk's local review suggestion and Gmail's own Spam label. No real message or sender is shown.